

Muhammad Ahamd

25280065

Task 3 – Transformations (TF2)

Objective

The objective of Task 3 was to implement coordinate frame transformations using the TF2 library in ROS 2. The task required broadcasting both a dynamic and a static transform and verifying the frame structure using the TF visualization tool.

Implementation

Dynamic Transform

A Python node was implemented to broadcast a transform from the world frame to a virtual_robot frame. The virtual robot was programmed to move in a circular trajectory with a radius of 2 meters. The transform was continuously updated to reflect the changing position of the virtual robot over time.

Static Transform

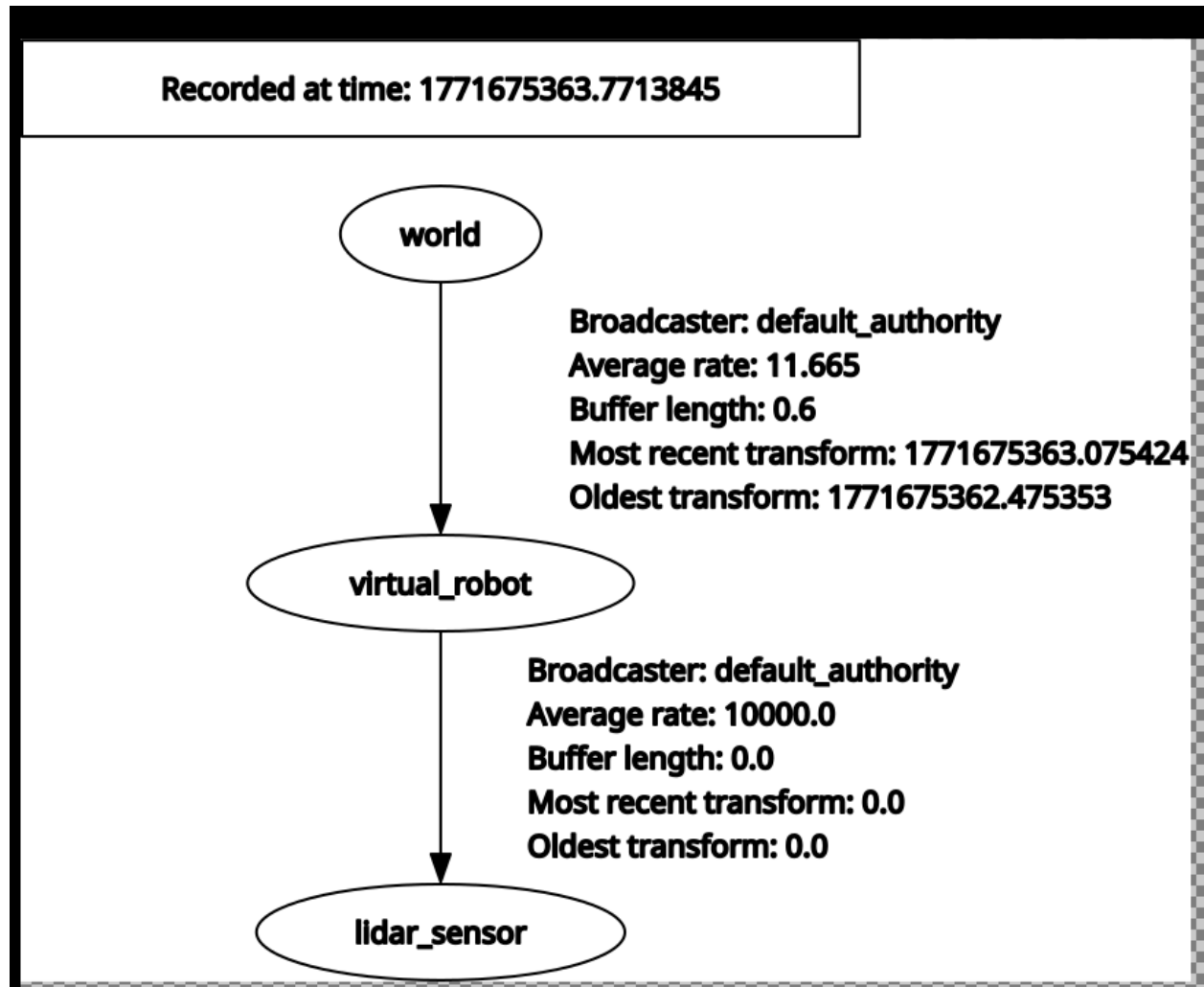
A static transform was defined between the virtual_robot frame and a lidar_sensor frame. The lidar frame was given a fixed offset relative to the virtual robot to simulate a sensor mounted on the robot body.

Verification

The launch file was executed to run both the static and dynamic nodes simultaneously. The TF tree was visualized using the TF viewer tool. The resulting frame structure correctly displayed:

world → virtual_robot → lidar_sensor

This confirmed that both the static and dynamic transformations were functioning as required.



Task 4 – Mapping (Approach & Plot)

Objective

The objective of Task 4 was to:

1. Move the TurtleBot3 Waffle Pi toward an object in Gazebo.
2. Circle around the object while maintaining a fixed distance using LIDAR data.
3. Record the object's position.
4. Generate a 2D plot representing the object's shape.

Simulation Setup

An empty Gazebo world was launched using the TurtleBot3 Waffle Pi model. A cube or sphere was inserted manually into the environment to serve as the object to be mapped.

Approach Phase

The robot used front LIDAR readings to detect the object and move toward it. If the object was beyond a defined distance threshold, the robot moved forward. Once the robot reached the specified distance, it stopped approaching and transitioned to the circling behavior.

Circling Phase (Wall Following)

During the circling phase, the robot used right-side LIDAR readings to maintain a constant distance from the object. A proportional control strategy was applied to adjust the robot's angular velocity dynamically. The object was kept approximately at 90 degrees relative to the robot to ensure smooth wall-following behavior.

The circular motion was not hardcoded; instead, it was fully controlled using real-time LIDAR feedback.

Mapping and Plotting

As the robot circled the object, detected obstacle points were recorded. These points were transformed from the robot's local coordinate frame into the world coordinate frame using the robot's pose.

After completing one full circle, the collected points were plotted in 2D. For a spherical object, the resulting plot forms a circular distribution of points.

Issue Encountered

During the execution of Task 4, significant issues were encountered with Gazebo:

- The TurtleBot3 model did not consistently spawn.

- Due to these simulation issues, the complete mapping demonstration could not be reliably finalized. However, the required logic for approach, wall-following, transformation, and plotting was implemented.



- Dynamic and static transforms were successfully implemented and verified.
- LIDAR-based approach and wall-following behavior were developed.
- Coordinate transformations from robot frame to world frame were applied.
- A 2D mapping logic was implemented.

Although Gazebo simulation instability prevented a fully consistent final demonstration, all required components were developed according to the task specifications.